9th Class • Punjab Board • PECTAA New Syllabus

Chapter 4: Computational Thinking

Complete solved exercise — decomposition, algorithms, flowcharts, pseudocode, dry runs and LARP.

Chapter 4 • Complete official exercise • 10 MCQs + 8 short + 5 long questions

What this chapter is about

- **Computational thinking** is a systematic way of solving problems. Its four key techniques are decomposition, pattern recognition, abstraction and algorithm design.
- An **algorithm** can be written as pseudocode or drawn as a flowchart. Both must be checked with sample inputs through a dry run.
- **LARP** means Logic of Algorithms for Resolution of Problems. It provides an interactive environment for writing and executing algorithms and flowcharts.
- Debugging distinguishes **syntax**, **runtime** and **logical** errors. Good algorithms are correct, finite, clear and efficient in time and memory.

Coverage: Chapter 4 corresponds to Unit 7 Computational Thinking in the twelve-unit textbook. The April 2026 nine-unit exercise is kept in book order; an official-unit grade algorithm is included as extension practice in Section D.

studysapiens.co

A. Multiple Choice Questions

1. Which of the following best defines computational thinking?

(a) A method of solving problems using mathematical calculations only.

(b) A problem-solving approach that employs systematic, algorithmic, and logical thinking.

(c) A technique used exclusively in computer programming.

(d) An approach that ignores real-world applications.

Answer: (b) A problem-solving approach that employs systematic, algorithmic, and logical thinking. — Computational thinking organises a problem and its solution logically so that the steps can be carried out by a person or computer.

2. Why is problem decomposition important in computational thinking?

(a) It simplifies problems by breaking them down into smaller, more manageabl parts.

(b) It complicates problems by adding more details.

(c) It eliminates the need for solving the problem.

(d) It is only useful for simple problems.

Answer: (a) It simplifies problems by breaking them down into smaller, more manageabl parts. — Smaller sub-problems are easier to understand, solve and test; their solutions are then combined.

3. Pattern recognition involves:

(a) Finding and using similarities within problems
(b) Ignoring repetitive elements
(c) Breaking problems into smaller pieces
(d) Writing detailed algorithms

Answer: (a) Finding and using similarities within problems — Recognising repeated features lets us reuse known ideas instead of solving every case from the beginning.

4. Which term refers to the process of ignoring the details to focus on the mainidea?

(a) Decomposition
(b) Pattern recognition
(c) Abstraction
(d) Algorithm design

Answer: (c) Abstraction — Abstraction hides irrelevant detail and keeps only the information needed for the current solution.

5. Which of the following is a principle of computational thinking?

(a) Ignoring problem understanding
(b) Problem simplification
(c) Avoiding solution design
(d) Implementing random solutions

Answer: (b) Problem simplification — Simplifying a problem makes its structure and possible solution clearer.

6. Algorithms are:

(a) Lists of data
(b) Graphical representations
(c) Step-by-step instructions for solving a problem
(d) Repetitive patterns

Answer: (c) Step-by-step instructions for solving a problem — An algorithm is a finite, ordered sequence of clear instructions that solves a problem.

7. Which of the following is the first step in problem-solving according to computational thinking?

(a) Writing the solution
(b) Understanding the problem
(c) Designing a flowchart
(d) Selecting a solution

Answer: (b) Understanding the problem — The inputs, outputs, constraints and goal must be understood before a useful solution can be designed.

8. Flowcharts are used to;

(a) Code a program
(b) Represent algorithms graphically
(c) Solve mathematical equations
(d) Identify patterns

Answer: (b) Represent algorithms graphically — Flowcharts use standard symbols and arrows to show the order and decisions in an algorithm.

studysapiens.com

9. Pseudocode is:

(a) A type of flowchart
(b) A high-level description of an algorithm using plain language
(c) A programming language
(d) A debugging tool

Answer: (b) A high-level description of an algorithm using plain language — Pseudocode combines plain language with programming-like structure without following one language's exact syntax.

10. Dry running a flowchart involves:

(a) Writing the code in a programming language
(b) Testing the flowchart with sample data
(c) Converting the flowchart into pseudocode
(d) Ignoring the flowchart details

Answer: (b) Testing the flowchart with sample data — A dry run traces each step manually with chosen inputs and records values and output to find mistakes.

B. Short Questions

1. Define computational thinking.

Computational thinking is a problem-solving process that uses logical, systematic and algorithmic methods to organise a problem and design a solution that a person or computer can carry out. It uses decomposition, pattern recognition, abstraction and algorithms.

2. What is decomposition in computational thinking?

Decomposition means breaking a large or complex problem into smaller, more manageable sub-problems. Each part can be understood and solved separately before the partial solutions are combined.

3. Describe abstraction in problem-solving.

Abstraction means focusing on the important features of a problem while ignoring details that are not needed for the current solution. For example, a route map shows roads and turns but hides the colour of every building.

4. What is an algorithm?

An algorithm is a finite sequence of clear, ordered instructions for solving a problem or completing a task. It must produce the required output for valid input and must eventually stop.

5. How does problem understanding help in computational thinking?

Problem understanding identifies the goal, inputs, expected outputs, rules and constraints before solution design begins. It prevents wasted work on the wrong problem and gives criteria for testing whether the algorithm is correct.

studysapiens.com

6. Why we use flow charts?

Flowcharts show an algorithm visually with standard symbols and arrows. They make sequence, decisions and loops easier to understand, communicate and dry-run before writing program code.

7. What is purpose of pseudocode.

Pseudocode describes an algorithm in structured plain language without depending on the strict syntax of a programming language. It helps plan the logic, find missing steps and convert the design into code later.

8. How do you differentiate between flowcharts and pseudocode?

Flowchart	Pseudocode
Graphical representation of an algorithm.	Text-based, high-level description of an algorithm.
Uses standard shapes and connecting arrows.	Uses plain-language statements with programming-like structure.
Shows branches and loops visually and is useful for explaining flow.	Usually quicker to write and edit and is closer to later program code.
A large diagram can take space and become difficult to redraw.	A long text description may be harder to visualise at a glance.

C. Long Questions

1. Define computational thinking and explain its significance in modern problem-solving.

Computational thinking is a disciplined way of understanding a problem and designing a step-by-step solution that can be communicated and tested. It is useful beyond programming because it turns a confusing situation into manageable, evidence-based work.

Technique	Meaning	Example: organising a school event
Decomposition	Break the problem into smaller parts.	Separate venue, invitations, food, budget and first aid.
Pattern recognition	Find similarities with current or earlier problems.	Reuse the checklist and registration method from last year's event.
Abstraction	Keep relevant details and hide unnecessary ones.	Use expected attendance and dietary needs, not every guest's favourite colour.

Technique	Meaning	Example: organising a school event
Algorithm design	Write ordered instructions that produce the solution.	List planning, booking, invitation, setup and evaluation steps in order.

• **Clarity:** It defines inputs, outputs and constraints, reducing misunderstandings.

• **Reusability:** A pattern or algorithm developed once can solve many similar problems.

• **Automation:** Clear steps can be converted into computer programs for fast, repeated work.

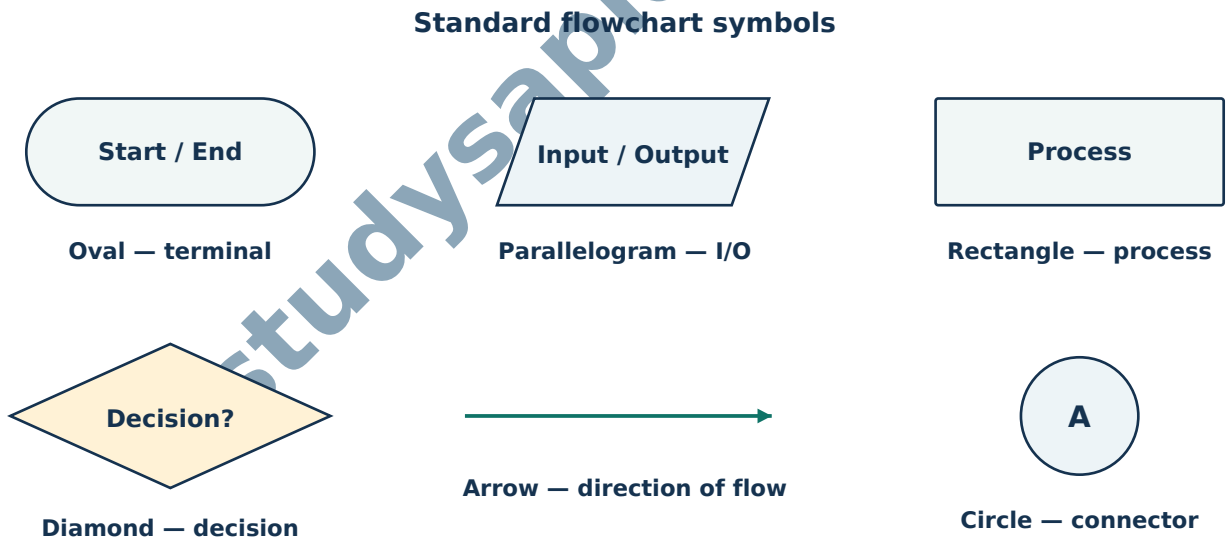
• **Testing and improvement:** A structured design can be dry-run, compared and corrected before implementation.

• **Cross-field use:** Doctors organise diagnoses, farmers plan irrigation, businesses forecast demand and students plan assignments with the same techniques.

2. Define flow chart and explain its different symbols.

A **flowchart** is a graphical representation of an algorithm in which standard symbols show actions, input/output and decisions, while arrows show the direction of execution. It is read from the start symbol along the connectors.

Figure 1 — Standard flowchart symbols and their meanings

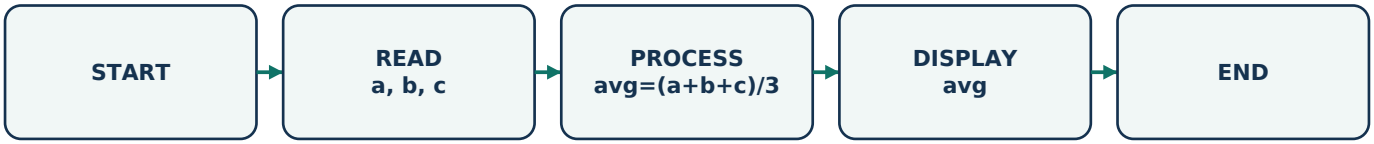


Symbol	Purpose
Oval / rounded terminal	Marks START or END.
Parallelogram	Reads input or displays output.
Rectangle	A process, calculation or assignment.
Diamond	A decision with two or more paths, commonly Yes/No.
Arrow / flow line	Shows which step is performed next.

studysapiens.com

Symbol	Purpose
Small circle connector	Joins parts of a larger flowchart without drawing a long crossing line.

Figure 2 — Sequence in an average algorithm



3. Explain pattern recognition in the context of computational thinking. How does identifying patterns help in problem-solving?

Pattern recognition means finding repeated elements, trends or similarities within a problem or between the current problem and earlier problems. These patterns reveal a general rule that can be reused.

1. Observe examples: Compare several cases and record what repeats.

2. Describe the pattern: Express the repeated feature as a rule, such as “every third record is a summary.”

3. Generalise: Check whether the rule applies to new cases of the same type.

4. Use the pattern: Reuse an existing algorithm or design one loop/rule instead of separate instructions for every case.

Example: When checking student attendance, marking 30 students one by one with 30 separately written blocks is unnecessary. Once the repeated pattern “read one name and mark present/absent” is recognised, a loop can repeat the same steps for every student.

Pattern recognition saves time, reduces repeated code and mistakes, supports prediction and helps us adapt proven solutions. The pattern must still be checked: a superficial similarity may not mean that the same solution is valid.

studysapiens.com

4. Describe what an algorithm is and fine example.

The printed phrase “fine example” is interpreted as “give an example.”

An algorithm is a finite, ordered set of unambiguous instructions that accepts defined input, performs operations and produces the required output. Its steps must be definite and possible to carry out. It forms the logical plan before a programming language is chosen.

Property	Meaning
Finiteness	It stops after a limited number of steps.

Property	Meaning
Definiteness	Every instruction has one clear meaning.
Input	It specifies the values supplied, if any.
Output	It produces at least one required result.
Effectiveness	Each operation is basic enough to perform.

Example: calculate the average of three numbers

Pseudocode

```
START
  READ a, b, c
  sum ← a + b + c
  average ← sum / 3
  DISPLAY average
END
```

Dry run with a = 6, b = 8 and c = 10

sum = 6 + 8 + 10 = 24

average = 24 ÷ 3 = 8

Output: 8

The algorithm has clear input, three definite processing/output steps and a finite END, so it can now be implemented in any suitable programming language.

5. What is LARP? Discuss its importance in learning and practicing algorithms.

LARP stands for **Logic of Algorithms for Resolution of Problems**. It is an interactive learning environment in which students write algorithms, draw related flowcharts and execute them with different inputs to observe the results.

• **Connects logic and output:** Learners see how each instruction and decision changes the result instead of only memorising definitions.

• **Simple syntax:** Commands such as START, READ, WRITE, IF…THEN…ELSE and END let learners focus on logic rather than difficult programming syntax.

• **Immediate testing:** Different inputs reveal whether the algorithm works for ordinary and boundary cases.

• **Visual learning:** Flowchart symbols make sequence, decisions and loops visible; execution links the diagram to pseudocode.

• **Debugging practice:** Error messages and dry runs help learners find missing steps, undefined variables, invalid operations and incorrect conditions.

• **Confidence and experimentation:** Students can modify an algorithm safely and compare the old and new outputs.

LARP-style even/odd algorithm

```
START
  WRITE "Enter a number"
  READ number
  IF number % 2 == 0 THEN
    WRITE "The number is even"
  ELSE
    WRITE "The number is odd"
  ENDIF
END
```

For number = 7, number % 2 = 1, so the ELSE path displays “The number is odd”. Testing 8 as well confirms that both decision paths work.

D. More Important Questions and Official-Unit Extension

1. Official-unit extension: Write an algorithm to assign a grade based on marks: 90 and above A+; 80–89 A; 70–79 B; 60–69 C; below 60 F.

Grade algorithm — conditions are checked from highest to lowest

```
START
  READ marks
  IF marks >= 90 THEN
    grade ← "A+"
  ELSE IF marks >= 80 THEN
    grade ← "A"
  ELSE IF marks >= 70 THEN
    grade ← "B"
  ELSE IF marks >= 60 THEN
    grade ← "C"
  ELSE
    grade ← "F"
  ENDIF
  DISPLAY grade
END
```

Dry-run checks

Input marks	First true condition	Output
95	marks ≥ 90	A+
85	marks ≥ 80	A
72	marks ≥ 70	B
60	marks ≥ 60	C
59	None; ELSE	F

Testing from the highest threshold downwards prevents 95 from incorrectly receiving a lower grade. A complete program should also validate that marks lie in

the permitted range, such as 0–100.

2. What is a dry run, and how does it help with debugging?

A dry run means manually executing an algorithm with selected sample inputs. At every step, record the current variables, condition result, path taken and output. It checks both flowcharts and pseudocode before coding.

Error type	When / why it occurs	Example
Syntax error	A command or required structure is written incorrectly.	Missing ENDIF or using an invalid symbol.
Runtime error	The algorithm starts but meets an impossible operation during execution.	Dividing by zero or accessing an undefined value.
Logical error	The algorithm runs but its logic gives a wrong result.	Using marks > 80 instead of marks ≥ 80, so 80 receives the wrong grade.

Test normal, boundary and invalid cases—for example 90, 89, 80, 60, 59 and 0 for the grade algorithm. A dry run exposes the exact step where values first differ from the expected result.

3. Why are time complexity and space complexity important when comparing algorithms?

Time complexity describes how an algorithm's running time grows as input size increases. **Space complexity** describes how its memory requirement grows. Two algorithms may give the correct answer, but one may be much slower or need much more memory for large input.

For example, checking every item in a list once grows roughly in proportion to n items, while comparing every item with every other item grows roughly with n^2 comparisons. Complexity helps us select an efficient solution, but correctness must be established first.

studysapiens.com

Points to Remember

Computational thinking: decomposition + pattern recognition + abstraction + algorithm design.

Problem understanding: identify goal, inputs, outputs, rules and constraints before designing.

Algorithm: finite, clear, ordered and effective instructions that produce the required output.

Flowchart: graphical symbols and arrows. **Pseudocode:** structured plain-language text.

Dry run: trace sample inputs through every step, recording variables, decisions and output.

LARP: Logic of Algorithms for Resolution of Problems; an interactive environment for algorithm practice.

Errors: syntax = invalid form; runtime = failure during execution; logical = wrong result despite running.

Efficiency: time complexity concerns growth in running time; space complexity concerns growth in memory use.

MCQ key at a glance

Q	Key	Q	Key	Q	Key	Q	Key	Q	Key
1	b	2	a	3	a	4	c	5	b
6	c	7	b	8	b	9	b	10	b

How to attempt this chapter in the paper

• For a flowchart, use the correct symbol for every step and label both branches of each decision.
• For pseudocode, indent IF/ELSE blocks and check thresholds from highest to lowest where ranges overlap.
• Always dry-run at least one normal and one boundary case; show the actual variable values, not only the final answer.
• In a computational-thinking answer, explain all four techniques with one consistent example instead of giving disconnected definitions.

StudySapiens.com